Construire une équipe projet

RAPPORT DE TRAVAUX PRATIQUES

Séances 1 à 8 - Thèmes 1 à 4

Projet : Application web de gestion administrative des notes des étudiants

(Node.js / React)

Équipe projet

Yamdjieu Gagoum Urich
Abossolo Bertrand
Teme Manga Ronald
Ngono Obam Rosane
Ossom Essam Roger Junior

Enseignant du cours

M. METILI Donatien

Année académique 2026-2027

Introduction

Ce rapport rassemble les travaux pratiques du cours GSA 712 / GSI 712, intitulé « Construire une équipe projet ». Le cours vise à comprendre comment une équipe projet se compose, s'organise et fonctionne au quotidien. Il s'articule autour de quatre thèmes : les rôles et les compétences, la répartition des tâches et la gouvernance, la gestion des conflits et la communication, puis les outils collaboratifs.

Pour ces huit séances, nous avons travaillé sur un projet réel : une application web de gestion administrative des notes des étudiants, développée avec Node.js côté serveur et React côté interface. L'application doit couvrir l'authentification, le référentiel académique, la saisie des notes, le calcul des moyennes, les rattrapages, les dettes académiques et l'édition des relevés de notes. Le TP 3 fait exception : il s'appuie sur le cas fictif d'application de covoiturage étudiant proposé en travaux dirigés, et les TP 7 et 8 reprennent ses tickets.

La méthode retenue pour le cours est Scrum. Elle donne un cadre simple à notre organisation : trois rôles (Product Owner, Scrum Master, équipe de développement), des sprints courts et des rituels réguliers. Notre équipe travaille en sprints de deux jours, répartis sur trois sprints, et applique un protocole Git commun où chacun travaille sur sa propre branche.

Ce document ne traite que les travaux pratiques. Chaque TP est présenté de la même façon : l'objectif, la manière dont nous l'avons mené, les résultats obtenus et ce que nous en retenons. Une conclusion générale et la bibliographie terminent le rapport.

Présentation de l'équipe

Notre équipe compte cinq membres. Le tableau 1 rappelle le rôle de chacun dans le projet, son rôle dans Scrum et sa branche Git.

Tableau 1 : Composition de l'équipe

Membre	**Rôle dans le projet**	**Rôle Scrum**	**Branche Git**
Yamdjieu Gagoum Urich	Coordination, animation des Daily Scrums, validation des Pull Requests	Scrum Master	main
Abossolo Bertrand	Développeur backend (base de données, API REST, authentification, calculs)	Équipe de développement	dev/bertrand
Teme Manga Ronald	Développeur frontend (intégration des maquettes Figma en React)	Équipe de développement	dev/roger
Ngono Obam Rosane	Règles métier, composants visuels, jeux de données, rédaction	Product Owner	dev/rosane
Ossom Essam Roger Junior	Conception (MCD/UML), scénarios de recette et tests	Équipe de développement (qualité)	qa/ronald

TP 1 :Constituer notre équipe projet

Objectif

Former l'équipe, attribuer à chacun un rôle Scrum, rédiger une description de poste pour chaque rôle et présenter l'ensemble à l'oral.

Déroulement

Formation du groupe. Nous avons formé un groupe de cinq étudiants, ce qui correspond à la limite haute demandée (quatre à cinq). Ce choix tient au projet lui-même : il demande de la programmation côté serveur, de l'intégration d'interface, un travail de cadrage des règles métier, de la conception et des tests, et il faut en plus quelqu'un pour coordonner le tout.

Choix des rôles Scrum. Scrum ne connaît que trois rôles. Nous les avons attribués en tenant compte du profil de chacun :

- **Product Owner** : Ngono Obam Rosane, qui porte les règles métier (barèmes, seuils éliminatoires, rattrapages) et la vision du produit ;
- **Scrum Master** : Yamdjieu Gagoum Urich, qui anime les rituels, suit l'avancement et valide les fusions ;
- **Équipe de développement** : Abossolo Bertrand (backend), Teme Manga Ronald (frontend) et Ossom Essam Roger Junior (conception et tests).

Roger Junior ne code pas de fonctionnalité, mais son travail de conception et de recette fait partie de l'équipe de développement : il valide chaque livrable avant qu'il soit fusionné dans main.

Descriptions de poste. Nous avons résumé chaque rôle en une phrase.

Tableau 2 : Descriptions de poste en une phrase

Rôle	Description
Scrum Master (Urich)	Anime les Daily Scrums, protège le processus Scrum, lève les obstacles qui bloquent l'équipe et fusionne les Pull Requests validées.
Product Owner (Rosane)	Définit les règles métier, priorise le Product Backlog et prépare les maquettes, les jeux de données et la documentation du projet.
Développeur backend (Bertrand)	Conçoit et développe la base de données, l'API REST, l'authentification sécurisée et le moteur de calcul des notes.
Développeur frontend (Ronald)	Transforme les maquettes Figma en interface React et la connecte à l'API du backend.
Testeur et concepteur (Roger Junior)	Valide les diagrammes MCD et UML, écrit les scénarios de recette et teste chaque livrable avant la fusion.

Présentation orale. Chaque groupe disposait de trois minutes. Nous avons construit la nôtre en quatre temps : le projet en une phrase, les membres et leur rôle, la manière dont leurs compétences se complètent, puis nos règles de travail (une branche Git par personne, aucune modification directe de main, Daily Scrum quotidien). Nous avons noté les remarques de l'enseignant à la fin de la présentation pour les intégrer dans la suite du projet.

Ce que nous retenons

Mettre des mots précis sur chaque rôle évite les zones grises. Dès cette première séance, nous savions qui décide des priorités, qui coordonne et qui construit. Cela a rendu les séances suivantes plus simples.

TP 2 :Recruter et intégrer un nouveau membre

Objectif

Partir d'un manque de compétence dans l'équipe, rédiger une offre de poste interne, mener un entretien entre pairs, puis intégrer le nouveau membre à notre organisation Scrum.

Déroulement

Compétence manquante. En relisant les périmètres de chacun, nous avons remarqué que personne ne s'occupait du déploiement : mise en ligne de l'application, variables d'environnement, intégration continue. Le plan de sprints n'en parle pas non plus. Or une application qui ne tourne que sur nos machines n'est pas vraiment livrée. Nous avons donc retenu le profil « déploiement et intégration continue ».

Offre de poste interne. Voici l'offre que nous avons rédigée.

- **Intitulé** : membre chargé du déploiement et de l'intégration continue.
- **Rattachement** : équipe de développement, sous la coordination du Scrum Master.
- **Missions** : mettre en place un environnement de test accessible à toute l'équipe ; automatiser le lancement des vérifications à chaque Pull Request ; documenter la procédure de déploiement ; aider les développeurs à corriger les problèmes d'environnement.
- **Profil recherché** : bonne maîtrise de Git et GitHub, notions de Node.js et de React, habitude des lignes de commande, sens de la rigueur.
- **Qualités attendues** : fiabilité, capacité à expliquer simplement, esprit d'équipe.
- **Disponibilité** : présence aux Daily Scrums et aux fins de sprint.

Entretien croisé. Nous avons fait des entretiens de dix minutes par deux, avec inversion des rôles : Rosane a interrogé Bertrand, puis ils ont échangé leurs places, et ainsi de suite pour les autres binômes. Les questions portaient sur la motivation pour le poste, l'expérience avec Git, la manière de réagir face à un conflit de fusion et la disponibilité pendant les sprints. L'exercice demande plus de préparation qu'on ne le croit : il faut des questions précises, sinon l'entretien devient une simple conversation.

Intégration. Nous avons formalisé l'arrivée du nouveau membre en quatre points :

- présentation du projet, du cahier des charges et de l'architecture du dépôt (dossiers /backend et /frontend) ;
- lecture commune du protocole Git et des règles de nommage JSON ;
- désignation d'un parrain pour les premiers jours : Bertrand, qui connaît le mieux la partie serveur ;
- ajout du nouveau membre au tableau Scrum de l'équipe, avec un premier ticket de prise en main (installation du projet en local).

Ce que nous retenons

Recruter ne se résume pas à choisir quelqu'un : il faut d'abord savoir ce dont l'équipe manque. L'intégration compte autant que la sélection, car un nouveau membre sans parrain ni documentation met longtemps à trouver sa place.

TP 3 :Transformer les lots de travail en Sprint Backlog

Objectif

Créer un tableau Kanban, transformer les lots de travail en tickets détaillés, attribuer chaque ticket à un membre et estimer sa taille.

Déroulement

Le tableau. Nous avons créé un tableau sur Trello avec trois colonnes : À faire, En cours et Terminé. C'est le minimum, mais il suffit pour que tout le monde voie où en est le travail.

Le cas étudié. Pour ce TP, nous avons repris le cas fictif du TD : une application de covoiturage étudiant. Le TD proposait cinq lots de travail (authentification, recherche de trajet, réservation, messagerie, évaluations) et laissait un sixième à trouver. Nous avons ajouté la **publication de trajets** : sans elle, aucun conducteur ne peut proposer un trajet et la recherche ne renvoie rien.

Des lots aux tickets. Chaque lot a été découpé en tickets assez petits pour être terminés en un jour ou deux, avec un seul responsable. Nous avons aussi ajouté quelques tickets transverses : initialisation technique, modélisation, données de test, dépôt Git et scénarios de recette.

Attribution et estimation. Chaque ticket porte le nom de son responsable, choisi selon son périmètre habituel : le serveur et la base de données pour Bertrand, les écrans pour Ronald, les maquettes et les données pour Rosane, la conception et les tests pour Roger Junior, l'organisation pour Urich. L'estimation s'est faite à voix haute : chacun proposait une taille (S, M ou L), puis nous discutions quand les avis divergeaient. Un ticket très disputé révélait souvent une mauvaise compréhension de son contenu.

Tableau 3 : Sprint Backlog : tickets, responsables et estimations

N°	Ticket	Responsable	Sprint	Taille
T01	Initialiser l'API Express et la base de données (utilisateurs, trajets, réservations)	Bertrand	1	M
T02	Inscription et connexion sécurisées (bcrypt, JWT)	Bertrand	1	L
T03	API de publication et de gestion des trajets par le conducteur	Bertrand	1	L
T04	Écran de recherche de trajet (départ, arrivée, date)	Ronald	1	M
T05	Pages d'inscription et de connexion	Ronald	1	S
T06	Jeu de données de test (utilisateurs et trajets)	Rosane	1	S
T07	Maquettes des écrans de réservation et de messagerie	Rosane	1	M
T08	Modélisation MCD et UML du covoiturage	Roger Junior	1	S
T09	Création du dépôt GitHub et règles de collaboration	Urich	1	S

N°	Ticket	Responsable	Sprint	Taille
T10	Moteur de réservation (places disponibles, annulation)	Bertrand	2	L
T11	Messagerie entre passager et conducteur	Ronald	2	L
T12	Système d'évaluations (note et avis après le trajet)	Bertrand	2	M
T13	Scénarios de recette des lots réservation et messagerie	Roger Junior	2	M
T14	Interface d'évaluation et profil des utilisateurs	Ronald	3	L

Ce que nous retenons

Un ticket sans responsable ne se termine jamais, et un ticket trop gros reste indéfiniment « en cours ». Estimer ensemble nous a aussi forcés à parler du contenu de chaque tâche avant de la commencer.

TP 4 :Simuler un Daily Scrum

Objectif

Reformer l'équipe de la séance 1 et simuler un Daily Scrum : chacun répond en une minute à trois questions, le Scrum Master note les blocages et propose des actions, puis nous résumons la situation de chaque membre en une ligne.

Déroulement

Nous avons simulé le Daily Scrum du matin du deuxième jour du sprint 1. Urich a lancé la réunion en rappelant la règle : quinze minutes au maximum, debout si possible, et aucune discussion technique longue. Les sujets qui demandent du temps sont renvoyés à un échange à part. Chacun a répondu à trois questions : qu'ai-je terminé hier, que vais-je faire aujourd'hui, ai-je un blocage ?

Tableau 4 : Déroulement du Daily Scrum simulé

Membre	Hier	Aujourd'hui	Blocages
Bertrand	Initialisation d'Express, connexion à la base de données, modèles, authentification	Routes CRUD du référentiel académique et des étudiants	Veut s'assurer que les noms de champs JSON respectent le cahier des charges
Ronald	Routage React, protection des routes par rôle, début de la page de connexion	Écrans de gestion du référentiel pour l'administrateur	Attend les routes d'authentification pour connecter la page de connexion
Rosane	Jeu de données initiales, début de l'analyse des besoins	Vues de gestion des étudiants et des enseignants	Aucun
Roger Junior	Relecture du MCD et des diagrammes UML	Tests de la branche de Bertrand	Pas encore de comptes de test pour tester les rôles
Urich	Création du dépôt et rappel du protocole Git	Suivi de l'avancement, relais des blocages	Aucun

Blocages notés et actions décidées. Urich a noté trois blocages et a proposé une action pour chacun :

- Ronald attend les routes d'authentification : Bertrand les pousse sur sa branche avant midi et prévient le groupe ;
- Roger Junior n'a pas de comptes de test : Rosane livre le jeu de comptes (un administrateur, trois enseignants, dix étudiants) avant la fin de la matinée ;
- Bertrand veut vérifier ses noms de champs : Roger Junior les compare avec le cahier des charges et signale tout écart.

Résumé d'une ligne par membre.

- **Bertrand** : serveur et authentification en place, passe aux routes du référentiel.
- **Ronald** : routage prêt, en attente des routes d'authentification pour finir la connexion.
- **Rosane** : données initiales livrées, travaille sur les vues étudiants et enseignants.
- **Roger Junior** : conception relue, prêt à tester dès réception des comptes.
- **Urich** : coordonne et suit la résolution des trois blocages.

Ce que nous retenons

Une minute par personne est plus court qu'on le pense, et cela oblige à dire l'essentiel. Le Daily n'est pas un compte rendu au chef : il sert à repérer les blocages avant qu'ils ne fassent perdre une journée.

TP 5 :Résoudre un conflit d'équipe

Objectif

Jouer un désaccord sur la répartition des tâches entre deux membres, appliquer la méthode DESC, faire médiatiser la situation par le Scrum Master, puis débriefer comme dans une rétrospective de sprint.

Le scénario

Nous avons choisi un conflit qui peut vraiment arriver dans notre équipe. D'après le plan, Rosane développe des composants React statiques à partir des maquettes, tandis que Ronald intègre ces mêmes maquettes en React. Les périmètres se touchent. Dans notre scénario, les deux ont commencé le même composant de liste des étudiants, chacun sur sa branche. Ronald s'en rend compte et vient en parler à Rosane, avec Urich comme médiateur.

Application de la méthode DESC

Tableau 5 : La conversation de Ronald à Rosane selon DESC

Étape	**Ce que dit Ronald**
Décrire	« Rosane, hier nous avons chacun commencé le composant de liste des étudiants, sur deux branches différentes. »
Exprimer	« J'ai passé deux heures dessus et je me suis senti frustré en découvrant que ton travail existait déjà. J'ai aussi peur des conflits au moment de la fusion. »
Spécifier	« J'aimerais que tu livres les composants statiques dans un dossier dédié et que je m'occupe de les connecter à l'API. Et qu'on s'écrive avant de démarrer un nouveau composant. »

Étape	Ce que dit Ronald
Conséquences	« Si on s'accorde, on évite les doublons et les conflits, et on gagne du temps pour le sprint. Sinon, on perdra du temps à la fusion et on risque de se démotiver. »

Écoute et reformulation. Rosane a d'abord reformulé : « Tu veux dire que mes composants statiques et ton intégration se chevauchent, et que tu aimerais une règle claire pour ne plus travailler deux fois sur la même chose ? » Elle a ensuite expliqué son point de vue : d'après le plan, les composants statiques font partie de son périmètre, elle pensait donc bien faire. Elle a reconnu qu'elle aurait dû prévenir. Une fois que chacun s'est senti compris, la discussion est devenue constructive.

Rôle du Scrum Master. Urich n'a pas tranché à la place des deux membres. Il a veillé à ce que chacun finisse ses phrases, il a ramené la discussion sur les faits plutôt que sur les personnes, et il a aidé à formuler un accord écrit.

Accord trouvé.

- Rosane livre les composants statiques dans un dossier dédié de /frontend ;
- Ronald les récupère pour les connecter à l'API et gérer les routes ;
- avant de démarrer un nouveau composant, chacun l'annonce dans le canal de l'équipe ;
- Urich note la règle dans le guide d'organisation.

Débriefing façon rétrospective

- **Ce qui a bien fonctionné** : parler des faits, puis des ressentis ; la reformulation avant de chercher une solution ; l'absence de reproche personnel.
- **Ce qui peut être amélioré** : le conflit est né d'un périmètre flou. Il aurait été plus simple de le clarifier dès le début du sprint.
- **Action retenue** : en début de chaque sprint, vérifier ensemble les zones où deux périmètres se rejoignent.

Ce que nous retenons

Un conflit bien traité nous a rapprochés au lieu de nous éloigner. Il vient presque toujours d'un malentendu sur les responsabilités, et c'est là qu'il faut agir en premier.

TP 6 :Organiser la communication d'équipe

Objectif

Mettre en place un canal de communication, définir nos rituels, simuler une Sprint Review devant un « client » et rédiger le compte rendu de la réunion.

Déroulement

Le canal de communication. Nous avons simulé un espace Slack organisé en canaux thématiques, pour éviter que toutes les conversations se mélangent.

Tableau 6 : Canaux de communication de l'équipe

Canal	Usage
#general	Annonces, organisation, horaires des réunions
#backend-api	Changements de routes API ou de champs de base de données
#frontend	Échanges sur les écrans, les composants et l'intégration
#recette-qa	Branches prêtes à tester, bugs signalés, validations
#urgent	Blocages qui ne peuvent pas attendre le prochain Daily

Les rituels. Nous avons fixé les règles suivantes :

- un Daily Scrum de quinze minutes maximum, à heure fixe, animé par Urich ;
- toute modification d'une route API ou d'un champ de base de données est signalée tout de suite dans #backend-api, afin que le frontend adapte ses appels ;
- une urgence passe par #urgent, avec un appel si le message reste sans réponse ;
- un Sprint Planning au début de chaque sprint, et une Sprint Review suivie d'une rétrospective à la fin.

Simulation de la Sprint Review. Nous avons joué une Sprint Review de fin de sprint 1. Le « client » était joué par l'enseignant et par Rosane, en tant que Product Owner. L'équipe a présenté l'incrément du sprint : la connexion par rôle, les routes de gestion du référentiel académique et le jeu de données de test. La démonstration a été faite sur l'application qui tourne, pas sur des diapositives.

Compte rendu de la réunion

- **Réunion** : Sprint Review du sprint 1.
- **Participants** : Urich (animation), Bertrand, Ronald, Rosane, Roger Junior et l'enseignant, dans le rôle du client.
- **Ordre du jour** : démonstration de l'incrément, retours du client, préparation du sprint 2.
- **Points présentés** : authentification par identifiant unique avec redirection selon le rôle ; gestion des diplômes, semestres, UE, EC et groupes ; données initiales de la filière Génie Informatique.
- **Retours simulés du client** : le principe est validé ; il faut afficher clairement les messages d'erreur à la connexion.

- **Décisions** : le sprint 2 commence par la saisie des notes et le moteur de calcul ; le message d'erreur de connexion est ajouté au backlog.

Tableau 7 : Actions issues de la réunion

Action	**Responsable**
Ajouter un message d'erreur clair à la connexion	Ronald
Prévoir le renvoi d'un code d'erreur explicite côté API	Bertrand
Intégrer le retour du client dans le backlog du sprint 2	Rosane
Rédiger la suite des scénarios de recette	Roger Junior
Diffuser ce compte rendu à l'équipe	Urich

Ce que nous retenons

Des échanges courts et réguliers valent mieux qu'une longue réunion occasionnelle. Écrire le compte rendu dans la foulée nous donne une mémoire du projet : on retrouve plus tard ce qui a été décidé et par qui.

TP 7 :Mettre en place un tableau Kanban

Objectif

Créer un tableau dans Trello ou Jira, y importer les tickets du TP 3, documenter chaque ticket dans un espace partagé et échanger des bonnes pratiques avec un autre groupe.

Déroulement

Choix de l'outil. Nous avons gardé Trello. Le projet est court, l'équipe est petite, et un outil simple que tout le monde utilise vaut mieux qu'un outil puissant que personne n'ouvre. Jira aurait demandé plus de configuration que ce que le projet justifie.

Tableau et import des tickets. Le tableau garde les colonnes À faire, En cours et Terminé. Nous y avons importé les tickets du TP 3, avec pour chacun son responsable, sa taille et son sprint. Le tableau 8 montre son état tel que nous l'avons configuré pour la fin du sprint 1.

Tableau 8 : État du tableau Kanban en fin de sprint 1

Colonne	**Tickets**
À faire	T10, T11, T12, T13, T14
En cours	T07
Terminé	T01, T02, T03, T04, T05, T06, T08, T09

Documentation des tickets. Une carte Trello ne dit pas tout. Nous avons donc créé dans Notion une page par ticket, selon un modèle commun : identifiant et titre ; description ; responsable ; taille ; branche Git ; dépendances ; critères d'achèvement (definition of done). À titre d'exemple, voici le contenu de la page du ticket T02.

- **Titre** : inscription et connexion sécurisées (bcrypt, JWT).
- **Responsable** : Bertrand. **Taille** : L. **Branche** : dev/bertrand.
- **Description** : inscription d'un utilisateur par adresse e-mail et connexion avec mot de passe haché, qui renvoie un jeton d'accès.
- **Dépendances** : T01 (API et base de données).
- **Critères d'achèvement** : une adresse e-mail déjà utilisée est refusée ; la connexion renvoie un jeton ; les routes protégées refusent un utilisateur non connecté ; le code démarre sans erreur ; Roger Junior a validé la recette.

Échange de bonnes pratiques. Nous avons comparé notre tableau avec celui d'un autre groupe. Nous avons retenu trois pratiques : donner à chaque ticket un titre qui commence par un verbe d'action, ne jamais laisser un ticket sans responsable, et mettre à jour le tableau pendant le Daily plutôt qu'en fin de journée.

Ce que nous retenons

Un tableau Kanban n'a de valeur que s'il reflète la réalité. Les pages de documentation évitent aussi que la connaissance d'un ticket reste dans la tête d'une seule personne.

TP 8 :Construire un tableau de bord de suivi

Objectif

Créer un canal de communication dédié, choisir trois indicateurs simples, construire un tableau de bord dans un tableur et l'ajuster lors d'un bilan intermédiaire.

Déroulement

Canal dédié. Nous avons ouvert un canal #suivi-sprint où Urich publie le tableau de bord après chaque Daily. Le tableau est ainsi visible de tout le monde, sans avoir à le demander.

Trois indicateurs. Nous avons gardé trois indicateurs, faciles à calculer et à comprendre : les tâches réalisées, les tâches restantes et la vélocité. Pour mesurer la vélocité, nous avons donné un nombre de points à chaque taille : S vaut 1 point, M vaut 2 points et L vaut 3 points.

Construction du tableau de bord. Nous avons utilisé un tableur avec deux feuilles. La première liste les tickets avec leur sprint, leur responsable, leur taille, leurs points et leur état. La seconde est la synthèse, calculée par formules : par exemple NB.SI pour compter les tickets « Terminé », et SOMME.SI pour additionner les points de ces tickets. Les valeurs du tableau 9 sont celles que nous avons saisies pour la simulation, à la fin du sprint 1.

Tableau 9 : Tableau de bord à la fin du sprint 1 (valeurs de simulation)

Indicateur	Tickets	Points
Tâches réalisées	8	14
Tâches restantes	6	15
Total du projet	14	29
Vélocité du sprint 1	—	14

Bilan intermédiaire et ajustements. Nous avons présenté le tableau de bord à l'équipe et recueilli ses retours. Deux ajustements ont été décidés :

- ajouter une colonne « Bloqué » dans la feuille des tickets, pour voir d'un coup d'œil ce qui dépend d'un autre membre ;
- afficher aussi les points par membre, afin de repérer une surcharge avant qu'elle devienne un problème.

Avec une vélocité de 14 points et 15 points restants (dont deux pour le ticket T07, encore en cours), le plan des sprints 2 et 3 nous a semblé réaliste. Nous garderons un œil sur le sprint 2, qui contient plusieurs tickets de taille L.

Ce que nous retenons

Un bon tableau de bord reste simple. Trois chiffres bien suivis nous renseignent mieux qu'un tableau chargé que personne ne regarde. S'il n'est plus consulté, il faut le simplifier : c'est un sujet idéal pour une rétrospective.

Conclusion générale

Ces huit séances de travaux pratiques nous ont fait parcourir tout le cycle de vie d'une équipe projet : sa constitution, son organisation, sa vie quotidienne, la résolution de ses tensions et les outils qui la soutiennent.

Plusieurs idées reviennent d'un TP à l'autre. D'abord, la clarté des rôles et des périmètres évite la plupart des problèmes, et le conflit du TP 5 en est la meilleure preuve. Ensuite, des rituels courts et réguliers (Daily Scrum, Sprint Review, rétrospective) maintiennent l'équipe alignée sans l'épuiser. Enfin, les outils comme Trello, Notion, Slack ou un tableur ne servent que s'ils sont simples et réellement utilisés.

Ces travaux nous ont surtout appris que la technique ne suffit pas : un projet de développement avance grâce à la manière dont les personnes travaillent ensemble. Nous allons maintenant appliquer ces pratiques pour mener notre application de gestion des notes jusqu'à la soutenance.

Bibliographie

Moutot, J.-M. et Autissier, D., *Méthode de conduite de projet*, Dunod, 2021.

Schwaber, K. et Sutherland, J., *The Scrum Guide*, 2020.

Hourst, B., *Manager une équipe projet*, Eyrolles, 2018.

Lencioni, P., *Optimisez votre équipe*, Un Monde Différent, 2005.

Atlassian, documentation officielle de Jira et Confluence, atlassian.com.